Machine Learning: Assignment 2

Bagging, Boosting and Bias Variance on Face Recognition

Total: 100 marks Individual assignment Report: CVPR format, maximum 8 pages

Due: Wednesday, 21 October 2026

Dataset and common setup. Use `fetch_olivetti_faces`: 40 identities, 10 images per identity, each 64×64. Use all numeric characters of your complete roll number, in order, as the seed for `numpy.random.default_rng`. Within each identity, randomly split the 10 images into 6 train, 2 validation and 2 test images. Save this split and use it everywhere. Use raw flattened pixels only. Do not use PCA or any other dimensionality reduction. You may use `DecisionTreeClassifier`, but bagging and boosting must be implemented by you. For the robustness experiment, create a second test set by setting the centred 16×16 block, rows and columns 24 to 39, to zero. Models must still be trained only on clean images.

Q1. Decision tree baseline and bagging from scratch 35 marks

(a) First establish a single tree baseline. Train `DecisionTreeClassifier` models with `max_depth`$\in \{2, 4, 8, 16, \texttt{None}\}$ and `min_samples_leaf`$\in \{1, 2, 4\}$. Select one setting using validation accuracy only and freeze it. Report train, validation and test accuracy, number of leaves, mean samples per leaf and the fraction of leaves containing only one training example. Explain what changes as depth increases. [9]

(b) Implement bagging yourself using bootstrap samples of the original training set size and the frozen tree setting from part (a). Use $B \in \{1, 5, 10, 25, 50, 100, 200\}$. For every B, report train, validation and test accuracy and the average number of distinct examples in one bootstrap sample. Compare the last quantity with its theoretical expectation. [9]

(c) For every B, compute out of bag accuracy, average pairwise disagreement between trees, and mean vote margin, where vote margin is the winning vote fraction minus the runner up vote fraction. Plot all three against B. Select one final ensemble size using the validation set and explain whether out of bag accuracy, disagreement and vote margin tell the same story. [8]

(d) Compare the frozen single tree and the final bagged model on both the clean and masked test sets. Report accuracy and prediction confidence for both. Find two faces where bagging corrects the tree and two where both are wrong. Show the tree prediction and bagging vote counts, and use these cases to explain what type of error bagging can reduce and what type it cannot. [9]

Q2. Multiclass boosting from scratch 35 marks

Implement SAMME AdaBoost yourself. For K classes, use

$$\alpha_t = \log\left(\frac{1-\epsilon_t}{\epsilon_t}\right) + \log(K-1), \qquad S_k(x) = \sum_t \alpha_t \, \mathbf{1}[h_t(x) = k].$$

Handle explicitly any weak learner that is not better than random multiclass prediction. For confidence use the normalised weighted vote $q_k(x) = S_k(x) / \sum_t \alpha_t$.

(a) Before using the face data, create a three class example with at most 10 samples. Log one complete SAMME round: initial sample weights, weak learner predictions, weighted error, α_t, and updated normalised weights. Your implementation on the face data must use the same update rule. [8]

(b) On Olivetti Faces, use weak learner depths $1, 2, 3, 4$ and $10, 25, 50, 100$ rounds. Select depth and rounds using clean validation accuracy only. For the selected model, plot weighted training error, learner weight, maximum sample weight and the total weight carried by the 10 heaviest training images over rounds. [9]

(c) Compare boosting and bagging using 25, 50 and 100 learners. Report train, validation and clean test accuracy and training time. Also inspect one training face that receives a very large boosting weight and use its prediction history to explain why boosting repeatedly returns attention to that sample. [9]

(d) Freeze the selected boosting model. Evaluate the single tree baseline, final bagged model and boosting model on the clean and masked test sets. Report accuracy and mean winning vote or probability for each. Find one face for which bagging and boosting respond differently to the missing region and explain the difference using tree disagreement and boosting sample weighting. [9]

Q3. Bias and variance as measured quantities 30 marks

Keep the final clean test set fixed. Create 30 new training sets by sampling with replacement, within each identity, six faces from that identity's original six training faces. Refit the selected single tree, the final bagged model and the selected boosting model on each resampled set. Do not tune again. For repetition r, let $p_r(x_i)$ be the class vector, $\bar{p}(x_i)$ its mean over repetitions and e_i the one hot target. For boosting use the normalised weighted vote vector. Compute

$$\text{Bias}^2 = \frac{1}{N}\sum_i \|\bar{p}(x_i) - e_i\|_2^2, \qquad \text{Variance} = \frac{1}{NR}\sum_{i,r} \|p_r(x_i) - \bar{p}(x_i)\|_2^2.$$

(a) Report measured bias squared and variance for the selected single tree, bagging and boosting models. Also report the mean and standard deviation of test accuracy across the 30 refits. [7]

(b) Repeat the calculation for single trees at one shallow depth, the selected depth and one much deeper depth. At the same three depths build 100 tree bagged ensembles. Quantify how depth changes bias and variance, and how much variance bagging removes at each depth. [8]

(c) Determine whether the deepest tree is necessarily the depth at which bagging gives the largest improvement in test accuracy. Your answer must use your measured bias, variance, leaf occupancy and disagreement results rather than a generic statement about the bias variance tradeoff. [7]

(d) End with one table for the selected single tree, bagging and boosting models containing clean test accuracy, masked test accuracy, bias squared and variance. Using this table together with your earlier plots, explain which model is most stable, which is most sensitive to the missing region, and whether the model with the lowest variance is necessarily the model with the best test accuracy. [8]

Submission. Submit one CVPR two column PDF, maximum 8 pages excluding references, and a GitHub repository containing the complete code, saved split, configurations, final logs, figure scripts, environment file and exact reproduction commands. Put your name, roll number, repository link and final commit hash on page 1. Any student may be asked to explain or rerun any part of the submission.

Generative AI. Use of ChatGPT, Claude, Gemini, Copilot, Codex, Cursor or similar systems for coding, debugging, analysis, figures, writing, rewriting or editing is prohibited. Use of generative AI for any part of the submitted work will result in zero marks.